Sparse-View CT Reconstruction on the Mayo Dataset

Variational, End-to-End, and Generative Approaches

Group B – Computational Imaging Project

May 25, 2026

Abstract

This report studies sparse-view computed tomography (CT) reconstruction as a computational imaging inverse problem. The same degraded Mayo CT measurements are used for all reconstruction methods, with images resized to 256×256, parallel-beam projection geometries with 180, 90, 60, and 45 views, and additive Gaussian measurement noise with relative level 0.005. Three methodological families are considered: Total p-Variation (TpV) regularization, an end-to-end Generalized ResUNet post-processing model, and a DiffPIR-based diffusion reconstruction method. At the current stage, the repository contains complete implementation evidence for TpV and Generalized ResUNet, while DiffPIR is kept as the selected generative method to be completed before final reporting. Quantitative evaluation is planned through PSNR and SSIM, complemented by visual comparison panels and error maps.

1 Introduction

Computed tomography is an imaging modality in which the internal attenuation map of an object is reconstructed from indirect X-ray projection measurements. In the two-dimensional setting considered in this project, the unknown image is denoted by $x \in \mathbb{R}^{256\times256}$ and the measured sinogram is obtained by applying a discrete CT projection operator. For a fixed number of projection angles, the forward model is written as

$$y^\delta = K_\theta x + e, \tag{1}$$

where K_θ denotes the parallel-beam Radon-type projection operator, y^δ is the noisy measured sinogram, and e is additive measurement noise. The reconstruction problem consists of estimating x from y^δ.

The task addressed in this project is sparse-view CT. In this setting, the number of projection angles is intentionally reduced. The project specifications require the configurations

$$n_\theta \in \{180, 90, 60, 45\}. \tag{2}$$

Reducing the number of views decreases the amount of measured data, but it also makes the inverse problem more ill-posed. In practice, this typically increases streaking artifacts and makes direct reconstruction less stable. Therefore, additional prior information is required either explicitly, by means of regularization, or implicitly, through learned image priors.

Mayo Dataset

The dataset used in the project is the Mayo CT dataset provided in the course material [1]. The local project archive contains PNG slices organized by patient folders. The processed split used by the current notebooks is summarized in Table 1. Each slice is resized to 256×256 and

Table 1: Mayo processed dataset organization used by the current notebooks.

Split	Patients	Slices
Train	8	2585
Validation	2	721
Test	1	327

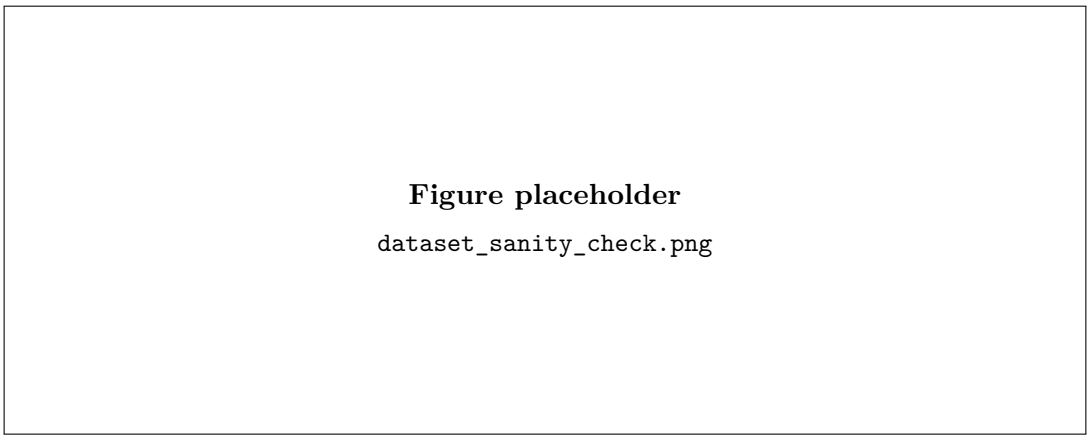


Figure 1: Suggested figure: one normalized Mayo CT slice and the corresponding noisy sparse-view sinograms for 180, 90, 60, and 45 projection angles.

normalized before the CT forward model is applied. The processed data are saved at patient level, preserving the relation between each patient identifier and its corresponding slices.

For each clean image x, four degraded sinograms are generated, one for each view configuration. The noise model is implemented in the measurement domain. If $y = K_\theta x$ is the clean sinogram, then the noisy data are

$$y^\delta = y + e, \qquad \frac{\|e\|_2}{\|y\|_2} \approx 0.005. \tag{3}$$

This construction ensures that all methods receive the same degraded observations. This point is essential for a fair comparison, because different noise realizations or different projection geometries would make the numerical results non-comparable.

Software Tools

The implementation is based on the IPPy library distributed with the course material. In this project, IPPy is used for the CT projection operator, filtered backprojection proxy, Chambolle–Pock TpV solver, U-Net architecture, and evaluation metrics. The main IPPy components are:

- `IPPy.operators.CTProjector`, which wraps the ASTRA-based CT forward and adjoint operators [2];
- `IPPy.solvers.FBP`, used only to compute the degraded image-domain proxy;
- `IPPy.solvers.ChambollePockTpVUnconstrained`, used for TpV optimization [3];
- `IPPy.nn.models.UNet`, used for the learned ResUNet post-processor [4];
- `IPPy.utilities.metrics`, used to compute PSNR and SSIM [5].

2 Selected Methods

2.1 Total p-Variation Regularization

The first selected method is a model-based variational reconstruction approach. The unknown image is estimated by minimizing a functional composed of a data-fidelity term and a Total p-Variation regularization term, following the general total-variation regularization principle [6]:

$$\widehat{x} = \arg\min_{x \geq 0} \frac{1}{2} \|K_\theta x - y^\delta\|_2^2 + \lambda R_p(x). \tag{4}$$

The data-fidelity term enforces consistency with the measured sinogram. The regularization term introduces prior information on the spatial structure of the image. In this project it is defined on the image gradient as

$$R_p(x) = \sum_i \|\nabla x_i\|_2^p, \qquad 0.1 < p < 0.5. \tag{5}$$

The exponent p is chosen in the non-convex range required by the project specifications. Since $p < 1$, the penalty promotes sparse image gradients more strongly than standard convex total variation. This is useful in CT reconstruction because anatomical structures often contain smooth regions separated by relevant edges.

The optimization is solved through a primal-dual Chambolle–Pock-type scheme. The solver used in IPPy applies a reweighted version of the TpV term. A smoothed gradient magnitude is introduced through the parameter η:

$$W(x) = \left(\frac{\sqrt{\eta^2 + |\nabla x|^2}}{\eta} \right)^{p-1}. \tag{6}$$

The weights depend on the current image estimate and modify the dual projection step associated with the gradient term. In this way, the non-convex TpV prior is handled by iteratively solving weighted primal-dual updates.

2.2 Generalized ResUNet Post-Processing

The second selected method is an end-to-end supervised learning method. The network does not operate directly on the sinogram. Instead, a filtered backprojection reconstruction is first computed from the sparse-view noisy sinogram, and this image is used as the degraded input to the neural network. The supervised mapping can be written as

$$\widehat{x} = f_\Theta(x_{\mathrm{FBP}}), \qquad x_{\mathrm{FBP}} = \mathrm{FBP}(y^\delta), \tag{7}$$

where f_Θ is the learned ResUNet and Θ denotes its trainable parameters.

The model is trained by minimizing the mean squared error between the network output and the clean Mayo slice:

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{j=1}^{N} \|f_\Theta(x_{\mathrm{FBP},j}) - x_j\|_2^2 . \tag{8}$$

The U-Net structure is appropriate for this task because it combines multiscale feature extraction with skip connections. The encoder extracts increasingly abstract features, while the decoder reconstructs a full-resolution image. Skip connections help preserve spatial information that may be lost during downsampling.

In this project, a single generalized ResUNet is trained across all four sparse-view configurations. Therefore, the training set contains FBP inputs generated from 180, 90, 60, and 45 views. This choice makes the network a general artifact-reduction model, exposed during training to different artifact intensities.

2.3 DiffPIR-Based Diffusion Reconstruction

The third selected methodological family is a generative method based on diffusion models, following the project requirement to adapt DiffPIR to sparse-view CT. At this stage of the repository, this method is not yet represented by a complete reconstruction notebook, so the present subsection defines the theoretical slot that must be completed once the implementation is finalized.

A diffusion model defines a prior over clean images through a gradual noising process and a learned reverse denoising process. In a generic denoising diffusion formulation, a clean image x_0 is progressively corrupted as

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{9}$$

where t is the diffusion step and $\bar{\alpha}_t$ is determined by the noise schedule. During reconstruction, the reverse process is guided not only by the learned image prior, but also by the physical CT data-consistency term. For sparse-view CT, the relevant consistency term is

$$\mathcal{D}(x) = \frac{1}{2}\|K_\theta x - y^\delta\|_2^2. \tag{10}$$

The role of the DiffPIR adaptation is to balance the generative denoising step with the requirement that the generated image remains compatible with the measured sparse-view sinogram. The strength of this correction is controlled by a parameter denoted here by ζ:

$$x \leftarrow x - \zeta \nabla_x \mathcal{D}(x) = x - \zeta K_\theta^\top (K_\theta x - y^\delta). \tag{11}$$

The value of ζ must be chosen carefully. If it is too small, the generative prior may dominate and produce anatomically plausible but data-inconsistent structures. If it is too large, the reconstruction may preserve the original sparse-view artifacts.

3 Implementation

3.1 Implementation of Total p-Variation

The TpV pipeline is implemented in `notebooks/01_TpV_reconstruction.ipynb`. The notebook loads the processed Mayo patient file and selects the central test slice. For each view setting, a CT projector is created as

$$\begin{aligned} K_\theta = \mathrm{CTProjector}\big(&\texttt{img\_shape} = (256, 256), \\ &\texttt{angles} = \mathrm{linspace}(0, \pi, n_\theta), \texttt{det\_size} = 256, \texttt{geometry} = \texttt{parallel}\big). \end{aligned} \tag{12}$$

The solver is then instantiated through `IPPy.solvers.ChambollePockTpVUnconstrained`. The current final-test parameters used in the notebook are

$$\lambda = 0.01, \qquad p = 0.35, \qquad \texttt{maxiter} = 150. \tag{13}$$

The solver is initialized from a zero image, because `starting_point=None` is passed to the IPPy call. The ground-truth slice is provided only for diagnostic computation of PSNR and SSIM during the iterations. The reconstruction is constrained to non-negative values inside the solver.

The notebook saves one visual panel per view configuration. Each panel contains the ground truth image, the TpV reconstruction, and the absolute error map. It also saves convergence plots containing the objective value, residual, PSNR, and SSIM curves.

3.2 Implementation of Generalized ResUNet

The learned method is implemented in `notebooks/02_ResUnet_reconstruction.ipynb`. The input to the network is generated in memory by applying `IPPy.solvers.FBP` to the saved noisy sinograms. The resulting FBP images are normalized per image and clamped to $[0, 1]$. These images are used only as degraded inputs, not as a separate project method.

The network is instantiated through `IPPy.nn.models.UNet` with one input channel and one output channel. The current configuration uses residual downsampling and upsampling blocks:

$$\begin{aligned} \texttt{middle\_ch} &= (32, 64, 128, 256, 512), \\ \texttt{n\_layers\_per\_block} &= 2, \\ \texttt{final\_activation} &= \texttt{sigmoid}. \end{aligned} \tag{14}$$

The sigmoid final activation constrains the output to the same normalized intensity range as the target images.

Training is supervised. For each patient batch, the notebook cycles through all four angle configurations. Therefore, the same clean target may be paired with different FBP inputs corresponding to different levels of angular undersampling. The optimizer is Adam with learning rate 10^{-3}, the batch size is 8, and the current training horizon is 20 epochs:

$$\texttt{learning\_rate} = 10^{-3}, \qquad \texttt{batch\_size} = 8, \qquad \texttt{epochs} = 20. \tag{15}$$

The checkpoint `weights/unet/resunet_generalized_latest.pt` stores the model state, optimizer state, loss history, validation loss history, and configuration.

3.3 Implementation of DiffPIR

The repository configuration marks DiffPIR as the selected generative method and excludes the hybrid PD-Net method for the two-student group. However, no complete DiffPIR reconstruction notebook is currently present in the repository. Therefore, this implementation subsection is intentionally written as a structured placeholder.

The final DiffPIR implementation should use the same Mayo processed tensors and the same IPPy CT projector as the other methods. For each view count, the algorithm should load the fixed noisy sinogram y^δ, use `IPPy.operators.CTProjector` for K_θ and $K_\theta^\top$, and apply the diffusion reconstruction loop with a data-consistency correction of the form in Eq. (11). The parameters that must be reported are the number of reverse diffusion steps, the data-consistency weight ζ, and the noise schedule used during sampling.

4 Results and Discussion

4.1 Hyperparameter Tuning

The project specifications require that each method has its main parameters selected heuristically and discussed. For TpV, the parameters are λ, p, and the number of Chambolle–Pock iterations. The parameter λ balances data consistency and regularization: a value that is too small may leave streak artifacts and noise, whereas a value that is too large may oversmooth image details and produce staircasing. The parameter p controls the non-convex sparsity of the gradient prior. The current selected value is $p = 0.35$, which lies inside the required interval $(0.1, 0.5)$.

For the ResUNet method, the relevant hyperparameters are the learning rate, batch size, number of epochs, base number of channels, and final activation. These parameters are selected by monitoring the training and validation MSE curves. The current configuration uses a learning rate of 10^{-3}, batch size 8, base channel count 32, and 20 epochs.

For DiffPIR, the tuning process must be completed after implementation. The parameters to be optimized are the number of diffusion sampling steps, the data-consistency multiplier ζ,

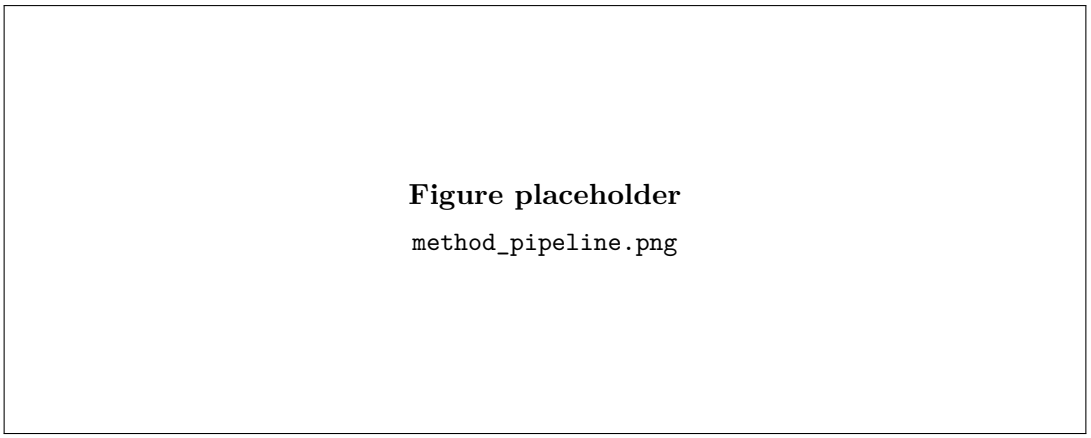


Figure 2: Suggested figure: complete experimental pipeline from Mayo slice to sparse-view sinogram, then to TpV, ResUNet, and DiffPIR reconstructions.

and the internal noise schedule. The expected trade-off is between generative image quality and physical consistency with the measured CT sinogram.

4.2 Evaluation Metrics

The quantitative evaluation uses PSNR and SSIM. The PSNR is computed from the mean squared error between reconstruction $\widehat{x}$ and ground truth x:

$$\mathrm{MSE}(\widehat{x}, x) = \frac{1}{N}\sum_{i=1}^{N}(\widehat{x}_i - x_i)^2, \tag{16}$$

$$\mathrm{PSNR}(\widehat{x}, x) = -20\log_{10}\left(\sqrt{\mathrm{MSE}(\widehat{x}, x)}\right), \tag{17}$$

where the images are assumed to be normalized in $[0, 1]$. SSIM is used as a structural similarity metric and is computed by the IPPy metric wrapper.

4.3 Quantitative Results

Table 2 is prepared for the final numerical values. The ResUNet entries are preliminary values printed by `notebooks/02_ResUnet_reconstruction.ipynb` on the central test slice. TpV values must be inserted from `notebooks/01_TpV_reconstruction.ipynb` or the final comparison notebook, and DiffPIR values must be inserted after the third method is implemented.

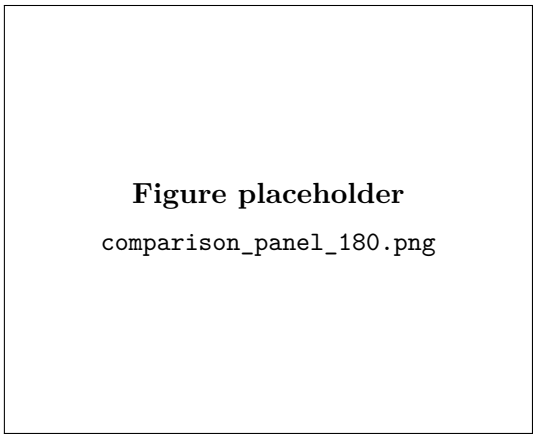
4.4 Visual Results and Critical Discussion

The visual comparison should include reconstructed images, absolute error maps, and quantitative labels for each method and view count. The most informative figures are expected to be: a dataset sanity check, TpV reconstruction panels, TpV convergence curves, the ResUNet training curve, ResUNet reconstruction panels, DiffPIR reconstruction panels, and a global method comparison panel.

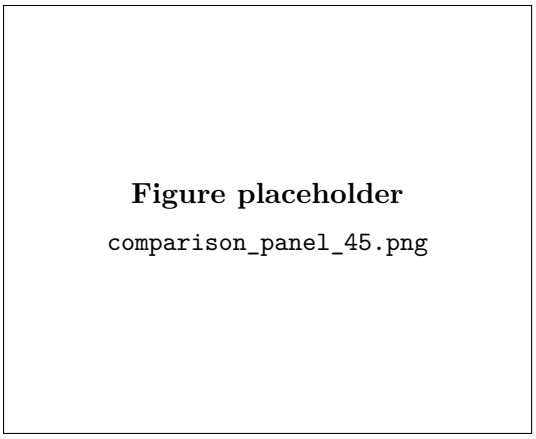
The final discussion should be written after the metric table and figures are available. The following points must be addressed. First, it should be verified whether the reconstruction quality decreases as the number of views is reduced from 180 to 45. Second, the TpV method should be discussed in terms of edge preservation, noise suppression, possible staircasing, and sensitivity to λ and p. Third, the ResUNet should be discussed in terms of artifact removal, possible oversmoothing, and dependence on the FBP proxy. Finally, the DiffPIR method should

Table 2: PSNR and SSIM comparison on the shared Mayo test slice.

Views	Method	PSNR (dB)	SSIM
180	TpV	**TODO:** fill	**TODO:** fill
180	Generalized ResUNet	13.8369	0.1259
180	DiffPIR	**TODO:** fill after implementation	**TODO:** fill
90	TpV	**TODO:** fill	**TODO:** fill
90	Generalized ResUNet	13.8369	0.1259
90	DiffPIR	**TODO:** fill after implementation	**TODO:** fill
60	TpV	**TODO:** fill	**TODO:** fill
60	Generalized ResUNet	13.8369	0.1259
60	DiffPIR	**TODO:** fill after implementation	**TODO:** fill
45	TpV	**TODO:** fill	**TODO:** fill
45	Generalized ResUNet	13.8369	0.1259
45	DiffPIR	**TODO:** fill after implementation	**TODO:** fill



(a) 180 views



(b) 45 views

Figure 3: Suggested figure: representative comparison panels for a high-view and a low-view sparse CT setting.

be evaluated by checking whether the generative prior improves visual quality without violating CT data consistency.

5 Conclusions

This report formulates sparse-view CT reconstruction on the Mayo dataset as an inverse problem and prepares a structured comparison of three methodological families. The TpV method provides an interpretable model-based reconstruction strategy based on explicit data fidelity and gradient-domain regularization. The Generalized ResUNet provides a supervised learned post-processing strategy trained from FBP proxy images to clean CT targets. The DiffPIR method is selected as the generative approach required by the project configuration, but its implementation and quantitative results still need to be completed.

The final version of the report should insert the missing PSNR and SSIM values, add the generated figures, and replace the DiffPIR placeholders with the exact implementation and experimental results. Once these elements are completed, the discussion will be able to compare the three approaches in terms of numerical accuracy, visual artifacts, computational cost, and robustness across sparse-view settings.

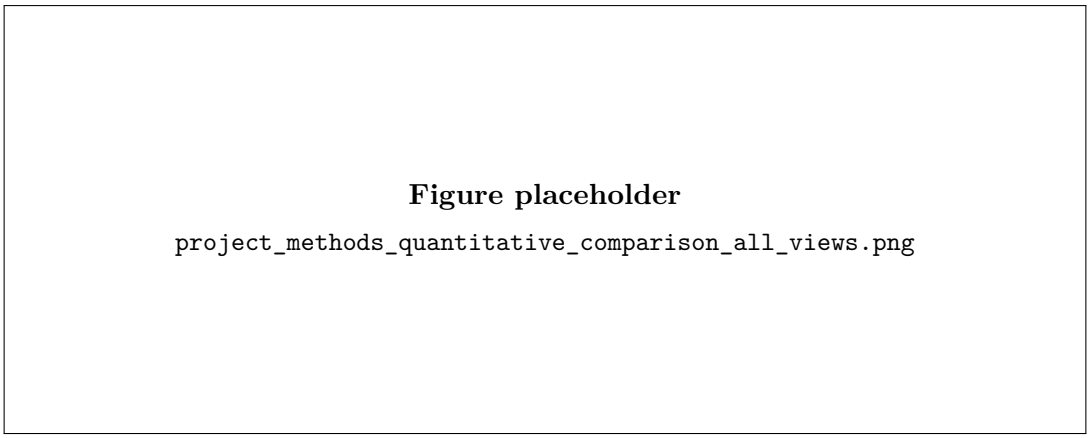


Figure 4: Suggested figure: aggregate PSNR/SSIM plot across view counts and methods.

References

[1] Cynthia H. McCollough. Tu-fg-207a-04: Overview of the low dose ct grand challenge. *Medical Physics*, 43(6):3759–3760, 2016.

[2] Wim van Aarle, Willem Jan Palenstijn, Jan De Beenhouwer, Thomas Altantzis, Sara Bals, K. Joost Batenburg, and Jan Sijbers. Fast and flexible x-ray tomography using the astra toolbox. *Optics Express*, 24(22):25129–25147, 2016.

[3] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.

[4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015.

[5] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[6] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4):259–268, 1992.